Deep Learning
CA3 Project Report

Title of Project:
CNN-LSTM Based Spatiotemporal Flood Evolution Forecasting
A comparison with simple baselines on the FloodCastBench dataset

Course Name:
Deep Learning

Programme:
B.Tech AIML, Batch 2024 to 2028, Semester 5

Submitted by:
Anhad Mahajan (PRN 24070126031, AIML A2)
Aman Sumeet Saxena (PRN 24070126030, AIML A2)

Faculty Name: Dr. Tanveer Ahmed and Dr. Ananya Bhattacharjee

Date of Submission: October 2026

Abstract

Floods change quickly, and a forecast of where the water will be in the next few hours helps people decide what to protect and where to move. In this project we built deep learning models that read the last few depth maps of a flood, together with the rain that has already fallen and the height of the ground, and predict the next water depth maps. The main model is a convolutional network followed by a convolutional LSTM (ConvLSTM). We trained and tested it on FloodCastBench, a public dataset of four simulated floods in Australia, the UK, Pakistan and Mozambique. On top of the forecaster we added a capsule network that rates how severe the flooding is, Grad-CAM heat maps, and written physics and severity rules.

We judged every model against simple reference forecasts, namely copying the last map and continuing the last change. Forecasts are scored with a skill number, where 0 means no better than copying the last map and 1 means perfect. In a notebook that forecasts whole maps 30 minutes ahead, our first model (Model A) beat copying in three of four tests, but it scored 0.15 to 0.35 in those tests, where the trend forecast scored 0.91 to 0.95 (and 0.99 in the fourth). A second model (Model B) that starts from the trend matched it but did not beat it. A larger study of about 310 training runs on a laptop, which forecasts up to 2.5 hours ahead, showed that the depth history carries almost all of the skill. A table with two inputs per cell reached 0.74 to 0.79, a fitted linear model of the depth history reached 0.79 to 0.82, and the best networks reached 0.81 to 0.87 (this is with the project's original input, which includes up to 30 minutes of future rain, and with past rain only the local CNN scores about 0.02 lower). Neighbouring pixels and a network about 140 times larger added nothing measurable. At equal training length the ConvLSTM was no more accurate than a plain CNN, although it was more stable when the start of a flood was hidden from training. Longer training gave a small gain (up to 0.024) and a larger batch with a higher learning rate a small loss (up to 0.026), which are only a few times the seed noise. We conclude that up to 2.5 hours a simple function of the recent depths of each cell does most of the work, and that the memory of the LSTM is still untested beyond that.

Keywords: flood forecasting, spatiotemporal forecasting, CNN-LSTM, ConvLSTM, FloodCastBench, capsule network, Grad-CAM, baseline comparison, skill score.

1. Introduction

1.1 Background and motivation

Floods damage homes, crops and roads every year, and the harm depends strongly on how early people are warned. Physics-based flood simulators solve the equations of water flow over the terrain and give detailed depth maps, but simulating a large flood can take days on a powerful GPU, as the authors of the FloodCastBench dataset report [1]. A trained deep learning model is much cheaper to run. It can produce a forecast in seconds, so it can act as a fast stand-in for the simulator [2, 3].

1.2 Problem statement

A flood is a sequence of maps. For each square cell of a grid we know the depth of the water at several moments, and we want the depth at the next moment. The model receives the last few depth maps, the rain that has already fallen, and the height of the ground. In the Colab notebook it receives four depth maps taken 30 minutes apart, so the oldest one is 1.5 hours old, and it predicts the map 30 minutes later. In the laptop study it receives five maps and predicts the next five, which reaches 2.5 hours ahead. In both parts the main test is a flood the model has never seen. The notebook also scores the last 15% of the training flood.

There is a catch. Over 30 minutes the water on most of the map hardly changes, so the simplest forecast, “the water stays where it is”, is already very accurate. A deep model is only useful if it does better than such simple forecasts. We therefore score every model with a skill number. A skill of 0 means no better than copying the last map, and 1 means a perfect forecast (the exact definition is in Section 3.3).

1.3 Objectives

1. Build a CNN-LSTM that forecasts flood depth maps, using TensorFlow and Keras.
2. Add a capsule network that rates the flood severity, Grad-CAM explanations, and written rules that check the network (the neuro-symbolic part of our original proposal).
3. Test the models on floods they have never seen and compare them with simple reference forecasts.
4. Run controlled experiments on the optimiser settings, the training length, the architecture and the inputs, to find out what drives the skill.
5. Report clearly which parts worked and which did not.

1.4 What we found, in short

- In the Colab notebook our first model (Model A) beat copying the last map in three of four tests. A trend forecast with one fitted number scored far higher. It

scored 0.91 to 0.99, while Model A scored 0.15 to 0.35 in the three tests it won and was worse than copying in the fourth.

- A second model (Model B) that starts from the trend forecast matched it but did not beat it. At 30 minutes ahead only 1% to 9% of the error of copying is left for a network to remove.
- In the larger laptop study the depth history carried nearly all of the skill. A fitted linear model of it already reaches 0.79 to 0.82. Neighbouring pixels and a network about 140 times larger added nothing measurable, and recurrence added no accuracy once the training length was equal.
- The capsule network did not beat a plain output layer once we used augmentation, and neither of them beat a simple threshold rule on the severity task.

1.5 Organisation of the report

Section 2 reviews related work. Section 3 describes the dataset, the models and the experiments. Section 4 gives the results and discusses them. Section 5 concludes and lists future work.

2. Literature review

2.1 Machine learning for floods

Mosavi et al. reviewed machine learning methods for flood prediction [2], and Bentivoglio et al. reviewed deep learning methods for flood mapping [3]. Together they show how widely machine learning is now used for floods. Kratzert et al. trained LSTM networks on rainfall and river flow records and showed that the network can learn the link between the two directly from data [4]. Much of this work predicts river flow or maps which areas are flooded. Our task is different, because we forecast the depth of the water on every cell of a map.

2.2 Convolutional and recurrent networks

A convolutional neural network (CNN) slides small filters over an image, so the same pattern detector works everywhere on the map [5]. A long short-term memory network (LSTM) reads a sequence step by step and keeps a memory that is controlled by gates [6]. Shi et al. combined the two ideas in the ConvLSTM, where the operations inside the LSTM are convolutions, so the memory is itself a map. They used it to forecast radar rainfall maps [7]. PredRNN is a later model that adds a second memory path to this idea [8]. A ConvLSTM looks like a natural choice for flood maps, because the water at one cell depends on the water around it. Section 4.3 tests whether this helps.

2.3 Flood forecasting benchmarks

FloodCast combines satellite observations, a hydrodynamic model and a neural network built on Fourier neural operators (FNO) to forecast floods over large regions [9, 10]. An FNO is a network that works on whole fields at once and can be used at different map sizes. The same group then released FloodCastBench, a dataset of four real floods that were re-created with a physics simulator at several map sizes [1]. Its benchmark trains U-Net, an image network with an encoder and a decoder [11], and two FNO models to forecast 19 steps from the first depth map. The benchmark does not include a recurrent model, and it compares the models with each other rather than with simple forecasts.

2.4 ConvLSTM models in flood studies

Xiao et al. built a ConvLSTM for urban flood depth under changing rainfall patterns and explored how well it extrapolates [12]. Xiang et al. used a ConvLSTM encoder-decoder with attention to forecast floods with a measure of uncertainty at the scale of a river basin [13]. These studies support the idea that convolutional-recurrent models suit flood data. These studies work at the scale of an urban area and of a river basin, while we test on floods from other countries.

2.5 Capsule networks, Grad-CAM and neuro-symbolic ideas

In a capsule network every output is a small vector. Capsules in one layer vote for capsules in the next layer, and votes that agree get more weight, which is called routing by agreement [14]. Grad-CAM (gradient-weighted class activation mapping) uses gradients to colour the parts of the input that mattered most for a prediction [15]. Neuro-symbolic AI combines a neural network with written rules that people can read [16]. Theory-guided data science adds scientific knowledge, such as conservation laws, to a learned model [17]. We use all three ideas as parts of our system.

2.6 Simple forecasts and skill scores

Forecasters judge a model against a simple reference, and the usual reference is the naive forecast that repeats the last observation [18]. A skill score measures how much of the reference error a model removes [19]. Studies in other fields show why this matters. Makridakis et al. compared statistical and machine learning methods on many time series and found that the statistical methods were more accurate [20], and Zeng et al. showed that a very simple linear model beat several Transformer models on long-term forecasting [21]. These results made us put simple forecasts at the centre of our comparison.

2.7 Summary and gap

Table 1 lists the main works. The flood studies above and the FloodCastBench benchmark compare deep models with each other. We did not find a study that tests convolutional-recurrent models on FloodCastBench and compares them with simple forecasts on floods the models have never seen. This report does that, and it also checks which parts of the CNN-LSTM matter.

Table 1: Main related works and how they connect to this project.

Work	What it does	Link to our project
Xu et al. 2025 [1]	FloodCastBench: four simulated floods, several map sizes, U-Net and FNO benchmarks	Our dataset
Xu et al. 2024 [9]	FloodCast: neural operator for large-scale flood forecasting	Earlier work of the same group
Shi et al. 2015 [7]	ConvLSTM for rainfall map forecasting	Basis of our forecaster
Kratzert et al. 2018 [4]	LSTM for rainfall-runoff modelling	Shows LSTMs can learn rain to flow links
Xiao et al. 2025 [12]	ConvLSTM for urban flood depth and its extrapolation	Closest ConvLSTM flood study
Xiang et al. 2025 [13]	ConvLSTM with attention for probabilistic flood forecasts	Recurrent flood forecasting at basin scale
Sabour et al. 2017 [14]	Capsule networks with routing by agreement	Our severity head
Selvaraju et al. 2017 [15]	Grad-CAM heat maps	Our explanations
Makridakis et al. 2018 [20]	Simple methods against machine learning in forecasting	Why we compare with simple forecasts
Zeng et al. 2023 [21]	A simple linear model against Transformers	Same reason

3. Methodology

3.1 Dataset description

FloodCastBench contains four real floods that the authors re-created with a physics-based simulator [1]. The flood maps are therefore simulated, not photographs or measurements. The simulator solves the two-dimensional shallow water equations in a simplified form. In plain words, it follows the basic rules of water flow: water runs downhill, rain adds water, and rough ground such as forest slows it down. It is built from real data for each region: a ground height map (FABDEM), a land cover map (Esri, made from Sentinel-2 images, with classes such as water, trees, crops and buildings) and satellite rainfall (GPM-IMERG). For each flood the dataset gives the water depth every 5 minutes, rain maps every 30 minutes, a ground height map and a land cover map. The rain and ground files are stored on a 30 m grid. Table 2 lists the four floods.

Table 2: The four floods. The dates are as given in the dataset paper, and the length is the length of the simulated record. "Maps" is the number of depth maps we keep, one every 30 minutes. Australia and Pakistan are the training floods, and the UK and Mozambique are the never-seen test floods.

Flood	Dates	Cell size	Map size (cells)	Length	Maps
Australia	20 Feb to 2 Mar 2022	30 m (and 60 m)	1073 × 1073	10 days	481
UK	4 to 7 Dec 2015	30 m (and 60 m)	170 × 275	3 days	145
Pakistan	18 to 31 Aug 2022	480 m	810 × 441	14 days	673
Mozambique	14 to 20 Mar 2019	480 m	151 × 138	6 days	289

In the Colab notebook we trained one 30 m model on Australia and tested it on the UK, and one 480 m model on Pakistan and tested it on Mozambique. The laptop study also used the 60 m maps of Australia and the UK. A 30 m cell and a 480 m cell behave very differently, so the two scales are never mixed.

How the coarser maps were made

According to the dataset paper, the 60 m maps of Australia and the UK are the 30 m simulation results resized to a coarser grid. For Pakistan, the authors shrank the terrain and rainfall to 480 m and ran the simulation again at that size. The paper does not say which method was used for the Mozambique 480 m maps, so we make no claim about it.

3.2 Pre-processing and inputs

1. **One map every 30 minutes.** Rain is given every 30 minutes and the water barely moves in 5 minutes, so we keep one depth map in six.
2. **One common grid.** The depth maps cover only part of the 30 m rain and ground files. We found where each depth map sits inside the 30 m grid and averaged the small cells inside each large cell. Two independent checks agreed within about one cell, namely rivers on the land cover map and valleys on the ground height map.
3. **Rain timing.** We measured that rain file *j* holds the rain that falls during the 30 minutes after depth map *j*. The rain files are in mm per hour, so half an hour of

rain would raise the average depth by 0.5 times the file value. On the active steps of the raw files, the measured rise was 0.494 to 0.502 times the file value in all four floods. Over a whole flood the totals differ by up to about 5% (Section 4.2).

4. **Rain settings.** A forecast may only use rain that has already fallen. The original loader of the project gave the last input frame up to 30 minutes of rain that had not yet fallen, which we call the *hybrid* input. We therefore test three rain settings: *causal* (past rain only), *hybrid* (the original input) and *forced* (the next five rain maps are also given, as if a rain forecast existed).
5. **Local ground height.** Water flows because a cell is higher or lower than its neighbours, not because of its height above the sea. In the Colab notebook we give the model the height of each cell minus the average height of the surrounding 15 by 15 cells. The laptop study used the height minus the average height of its 128 by 128 patch.
6. **Tiles.** The Australian map has over a million cells, so the models learn from small square pieces, which we call tiles or patches (64 by 64 cells in the Colab notebook and 128 by 128 in the laptop study). Because the networks only use small sliding windows, a trained network can afterwards run on a whole map.

3.3 Baselines and the skill score

We compare every model with simple reference forecasts. Most of them need no learning, and the others have very few parameters.

- **Copy the last map** (also called persistence). The forecast is the last observed depth.
- **Trend forecast.** Let d_t be the depth map at the last observed time and d_{t-1} the map 30 minutes earlier. The forecast adds a fraction β of the last change to the last map and is never below zero:

$$\hat{d}_{t+1} = \max\bigl(0,\ d_t + \beta\,(d_t - d_{t-1})\bigr)$$

  The single number β is fitted by least squares on the training part of the training flood only. It was 0.921 for the 30 m floods and 0.948 for the 480 m floods. A value below 1 means that the change is continued in a slightly damped way.
- In the laptop study we also used a fitted linear model of the depth history, linear models that add rain and terrain, a lookup table with two inputs (the current depth and the last change), and a multilayer perceptron (MLP), which is a small network that works cell by cell. The linear models and the MLP are given each cell's own depth history, rain and height, plus a 15 by 15 average of the depth and of the last change around the cell. The linear models are ridge regressions, which are least-squares fits with a small penalty on the size of the weights. Their inputs are rescaled to the same spread before fitting, because otherwise the penalty would push the weights of the very small depth changes to zero. The lookup table uses nothing but the two values of the cell itself.

The main score is the *skill*:

$$\text{skill} = 1 - \frac{\sum(\text{forecast} - \text{truth})^2}{\sum(\text{copy of the last map} - \text{truth})^2}.$$

A skill of 1 is a perfect forecast, 0 means no better than copying, and a negative value means worse than copying. A skill of 0.9 means that the forecast removes 90% of the squared error of copying. For the typical error in centimetres we use the root mean square error. In the laptop study the intervals are 95% block-bootstrap intervals. For these we resampled blocks of start times or tiles many times and recomputed the score each time, and the middle 95% of the results is the interval.

3.4 Model architecture

Figure 1 shows the whole system, and Figure 2 shows the forecaster.

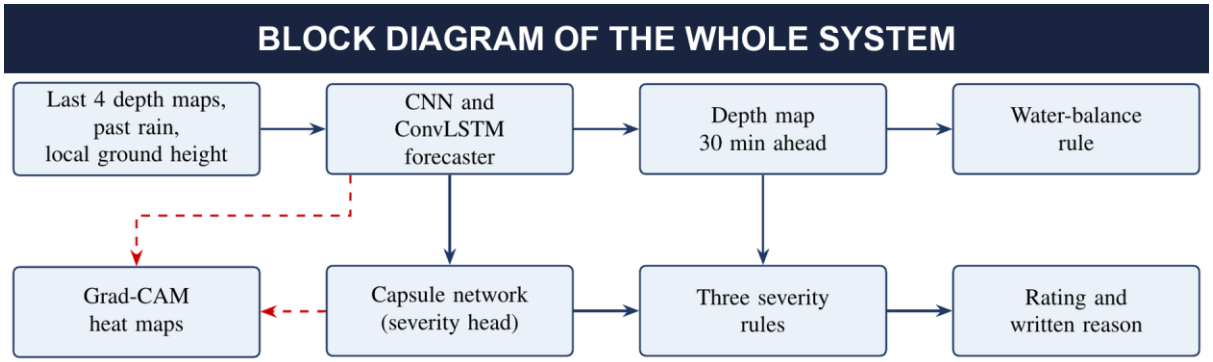


Figure 1: Block diagram of the system. The forecaster predicts the next depth map. The capsule network reads the same ConvLSTM memory and rates the flood severity. The water-balance rule corrects the total water in the forecast, and the severity rules check the capsule network's rating using the forecast map. Grad-CAM (dashed) explains both the forecaster and the capsule network.

The forecaster

Two 3 × 3 convolution layers with 16 filters are applied to each of the four input maps, so the network can pick out water edges, slopes and rain patches. A ConvLSTM layer with 32 filters then reads the four maps in time order and keeps a memory map of how the water has been moving. A small decoder turns the memory into the predicted *change* in depth. Predicting the change is easier than redrawing the whole map, because most of the map hardly changes in 30 minutes. The last layer starts at zero, so before training the model gives exactly its starting forecast, and training can only add to it. Table 3 lists the layers of Model A.

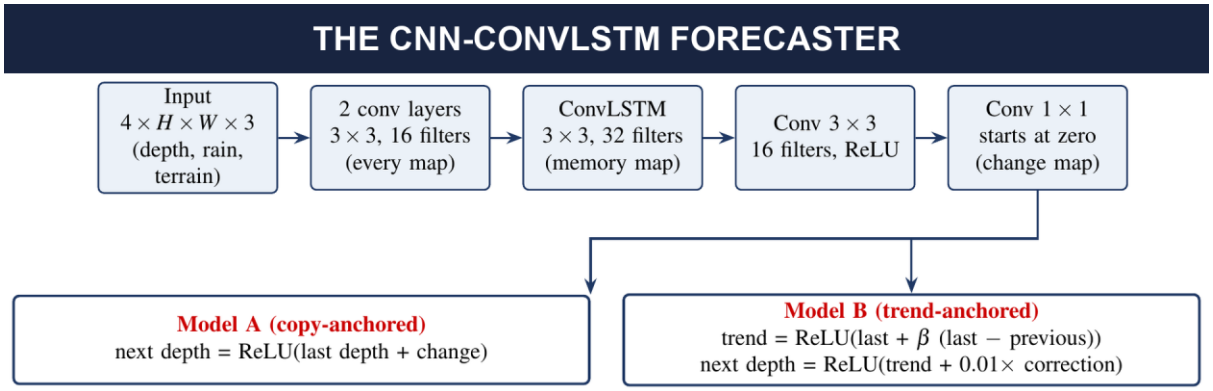


Figure 2: The CNN-ConvLSTM forecaster (62,833 parameters). The input is four maps of $H \times W$ cells with three layers each. ReLU sets negative values to zero, so the depth never goes below zero. Model A adds the predicted change to the last depth map. Model B adds a small correction, counted in centimetres, to the trend forecast, which is built from the last two depth maps. In both models the last layer of the network starts at zero.

Table 3: Layers of Model A. Model B has the same layers. It only changes the last step, where the correction is added to the trend forecast instead of to the last map.

Layer	Output shape (64 × 64 tiles)	Parameters
Input (depth, rain, local height)	4 × 64 × 64 × 3	0
Convolution 3 × 3, 16 filters, ReLU (on each map)	4 × 64 × 64 × 16	448
Convolution 3 × 3, 16 filters, ReLU (on each map)	4 × 64 × 64 × 16	2,320
ConvLSTM 3 × 3, 32 filters	64 × 64 × 32	55,424
Convolution 3 × 3, 16 filters, ReLU	64 × 64 × 16	4,624
Convolution 1 × 1, 1 filter (starts at zero)	64 × 64 × 1	17
Add to the starting forecast, then ReLU	64 × 64 × 1	0
Total		62,833

Table 4 lists all the models of this report. In the Colab notebook, Model A and Model B are called the first version and version 2.

Table 4: The models. Two large dense layers (of 16,384 × 128 weights each) hold about 93% of the parameters of the original CNN-LSTM.

Model	Description	Parameters	Used in
Original CNN-LSTM	Three strided convolutions, flatten, a dense layer of 128 numbers, an LSTM, and a decoder with a second large dense layer	about 4.5 million	Laptop study
Local CNN	One 5 × 5 convolution (16 filters) and a 1 × 1 convolution on features of the depth history, terrain and rain, no recurrence	4,101	Laptop study
Small ConvLSTM	ConvLSTM (8 filters, 3 × 3) at full resolution, then 3 × 3 and 1 × 1 convolutions	4,741	Laptop study
Heavy ConvLSTM	ConvLSTM encoder-decoder with skip connections and a local convolution path	652,918	Laptop study
Model A	CNN-ConvLSTM, copy-anchored (Table 3)	62,833	Colab notebook
Model B	Same network, trend-anchored	62,833	Colab notebook

The original CNN-LSTM was the first model of the project. It squeezes the features of every frame through one dense layer of 128 numbers before the LSTM, so it is hard for

small per-cell changes in depth to pass through it. This turned out to matter a lot (Section 4.3).

3.5 Severity branch, rules and Grad-CAM

Capsule network

The capsule head takes the ConvLSTM memory of the forecaster (kept frozen) and applies three strided convolutions. It then forms 256 primary capsules of 8 numbers each and passes them through three rounds of routing by agreement to four class capsules of 16 numbers each. The length of a class capsule says how sure the network is about that class. The loss is the margin loss of Sabour et al. [14], which pushes the length of the correct capsule above 0.9 and the others below 0.1, and rare classes get larger weights during training. In the laptop study we also trained a plain dense output layer on the same features as a control.

Severity labels

We used two labels. The earlier laptop study labelled a patch by its deepest cell, in bins of 0.3, 1 and 2 m. That label does not separate the classes outside Australia at 30 m. The Colab notebook labels a 64 × 64 tile by the share of its normally dry land that is under more than 30 cm of water: none (below 5%), minor (5 to 15%), moderate (15 to 30%) and severe (30% or more). Rivers and lakes are left out. This label gives a spread of classes in Australia, the UK and Mozambique. In Pakistan 698 of the 1,000 test tiles are severe, and in the UK none is.

Water-balance rule

Almost all of the rain stays on the ground in these simulations, so the average rise of the water over a map in 30 minutes should equal the rain in mm per hour times 0.5 hours, if no water crosses the edge of the map. The rule shifts the forecast so that its total change matches the rain of the last 30 minutes, which is the only rain that is known. Where the water goes is still decided by the network.

Severity rules

Rule 1 sets the rating to “none” if less than 1% of the land is flooded now and in the forecast. Rule 2 stops the rating from falling below the current level when the forecast is not less flooded than the present and more than 1 mm per hour of rain fell in the last hour. Rule 3 replaces the capsule rating by the rating of the forecast map if the two are more than one class apart. Every decision comes with a written reason.

Grad-CAM

Following Selvaraju et al. [15], we weight the ConvLSTM memory maps by the average gradient of the quantity we want to explain and keep the positive part. For the forecaster that quantity is the total predicted depth. For the capsule network it is the length of the chosen class capsule. In the laptop study we also ran a deletion test. It removes the cells

with the highest heat and checks that the forecast changes more than when random cells are removed.

3.6 Implementation and training

All models were written in TensorFlow and Keras and trained with the Adam optimiser [22]. The Colab notebook ran on a GPU runtime (T4 or L4 class) with Keras 3. The laptop study ran on a single CPU laptop with 15.5 GB of memory, using TensorFlow 2.21 and Keras 3.15. Table 5 gives the training settings. A pass is one run through all training data (one epoch).

Table 5: Training settings of the two experiment sets.

	Colab notebook (Models A and B)	Laptop study (fixed recipe)
Optimiser	Adam, learning rate 10^{-3}	Adam, peak learning rate 10^{-3}
Schedule and clipping	Constant rate	200-step warm-up, then cosine decay, gradient-norm clip 1.0
Loss	Mean squared error (Model A in m^2, Model B in cm^2)	Mean squared error divided by the error of copying the last map
Batch size	16	1
Training length	Up to 20 passes, early stopping after 4 passes without gain, best weights kept	30 passes, final weights used, no early stopping
Training data	4,000 random 64 × 64 tiles (600 for validation)	300 windows of 128 × 128 cells
Evaluation	Whole maps, 30 minutes ahead	Held-out tiles and never-seen flood, 5 maps in and 5 out
Seeds	42	1003, 1013 and 1023 (one per fold)

The capsule network of the Colab notebook was trained for up to 30 passes with a batch size of 32 and early stopping after 5 passes without gain. Its loss was the margin loss with class weights, so that rare classes count more.

3.7 Experimental design

The project has two parts. The first is a larger controlled study that we ran on a laptop to find out which parts of a CNN-LSTM really matter (experiment set II below). The second is a Google Colab notebook with our submitted implementation, which tests Models A and B on whole maps (experiment set I). The two parts use different models, patches and protocols, so their numbers cannot be compared, and Models A and B were not part of the laptop study.

Experiment set I (Colab notebook, whole maps)

Two models were trained, one per map scale. The 30 m model learned from the first 70% of the Australian flood, used the next 15% for early stopping, and was tested on the last 15% of Australia and on the whole UK flood. The 480 m model did the same with Pakistan and Mozambique. We compared Model A and Model B with copying, the trend forecast and the water-balance rule, and we trained the capsule network, the severity rules and Grad-CAM on top.

Experiment set II (laptop study)

Before the final runs we wrote down the protocol and six hypotheses. In plain words, they said that (H1) the models beat copying, (H2) they beat linear baselines, (H3) the depth history matters, (H4) recurrence matters, (H5) knowing more rain helps, and (H6) a stricter test gives similar scores. The strict test also hides a block of time from training, not only some tiles. The experiments were Australia to UK at 60 m (E60) and at 30 m (E30), and Pakistan to Mozambique at 480 m (E480). Each one used three folds. In each fold, a different third of the tiles of the training region was held out. We also used three rain settings and several models. The models produce all five forecast maps in one pass and do not feed their forecasts back as input.

After seeing the first results we added exploratory experiments on the optimiser setting, the training length, the number of windows, the random seed and transfer in the reverse direction. The large ConvLSTM was already part of the written plan. The linear baselines were also fitted a second time with rescaled inputs, after we found that the first fit had pushed the weights of the small depth-change inputs almost to zero. Together this made about 310 training runs. All deviations from the plan are listed in the project log, which we keep with the code.

Code and data

The data are public [1]. The data preparation, visualisation and model notebooks of the Colab part, and the code and the 310 result files of the laptop study, are kept in the project package.

4. Results and discussion

4.1 Experiment set I: whole-map forecasts 30 minutes ahead

Table 6 gives the skill of every forecast on the four test parts. The copy forecast has a typical error of 0.112, 0.520, 0.103 and 0.548 cm in the four tests, averaged over the whole map including dry cells.

Table 6: Skill against copying the last map, whole maps, 30 minutes ahead (single runs). A and B are Models A and B. "Rule" is the water-balance rule. Copy has skill 0 by definition.

Test	Trend	Copy + rule	A	A + rule	B	B + rule
Australia, last 15%	0.990	0.000	−1.449	−0.710	0.989	0.989
UK, never-seen flood	0.947	0.053	0.305	0.320	0.948	0.949
Pakistan, last 15%	0.908	0.142	0.345	0.404	0.908	0.901
Mozambique, never-seen flood	0.942	0.051	0.150	0.172	0.943	0.941

Model A beats copying in three of the four tests. But the trend forecast, which has no network at all, scores between 0.908 and 0.990 and is far ahead of Model A in every test (Figure 3). We checked this comparison in two ways. A separate script on the raw files reproduced the copy errors of the notebook to three decimals, and the same code run inside the notebook gave the same trend scores.

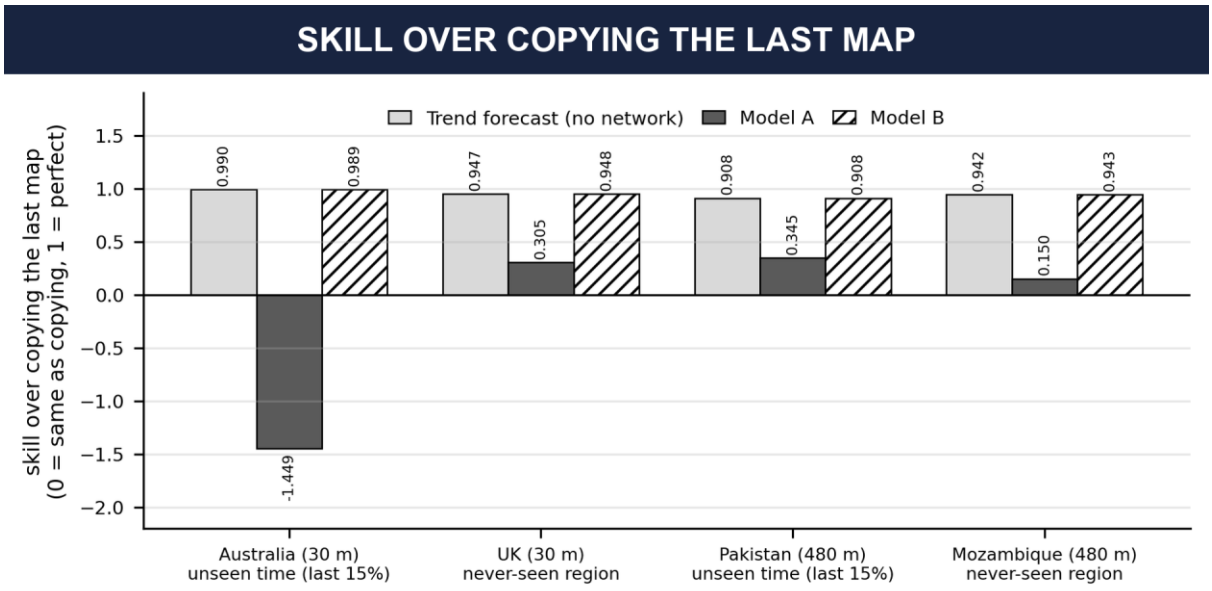


Figure 3: Skill over copying the last map for the trend forecast (no network), Model A and Model B. Bars above the black line beat copying and 1 is perfect.

Late Australia

Model A scored −1.449 on the last 15% of the Australian flood, so it was worse than copying. At that stage the water hardly moves, and copying is wrong by only about one millimetre. We first thought that the model had learned only from rising water and never saw water draining away. That explanation does not hold. In 73 of the 331 training steps

the average depth of the Australian map fell, so falling water was in the training data. In the last 15% the average depth still rises slightly in every step. The trend number was fitted on the same training steps and scores 0.990 on that last part. Figure 4 shows that the error of Model A stays between 0.16 and 0.21 cm while the error of copying falls to about 0.08 cm. The first row of Figure 5 (hour 204, the start of this last part) shows a faint excess of water over most of the map. We think Model A added a little water almost everywhere, but we did not measure this further.

Model B

Model B starts from the trend forecast and learns a small correction. Its skill is within 0.001 of the trend forecast in every test. Training lowered the error on the validation tiles only a little, by about 1% for the 30 m model (0.1885 to 0.1868 cm) and by about 5% for the 480 m model (0.0243 to 0.0230 cm). So Model B matches the trend forecast but does not beat it. We changed two things at once when we built it, the starting point and the unit of the correction, so we cannot say which one made the difference. At 30 minutes the trend forecast already removes 91% to 99% of the squared error of copying. That leaves about 1%, 5%, 9% and 6% of it for any model to remove in the four tests.

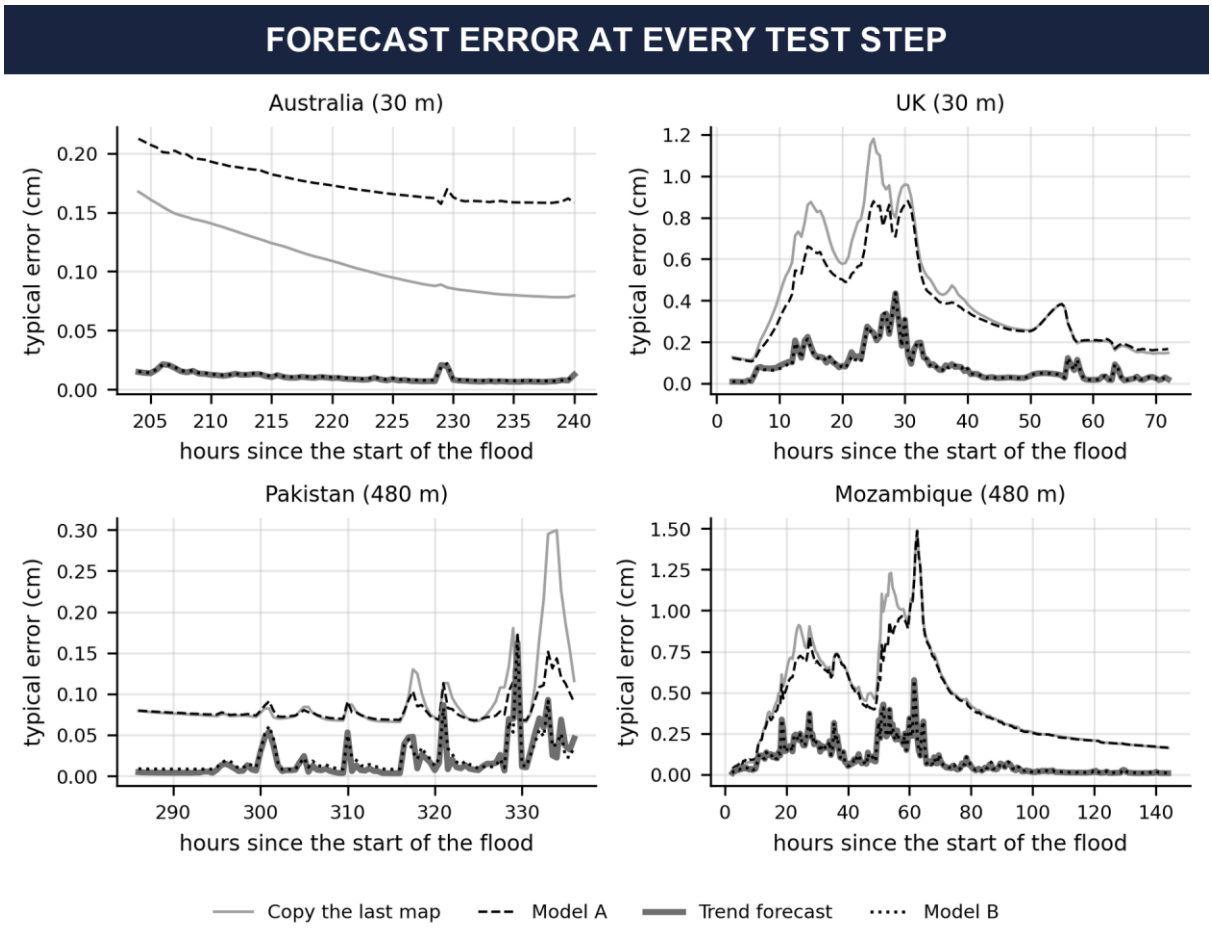


Figure 4: Error at every step of the test period (lower is better). Model A is above copying in late Australia. The trend forecast and Model B lie almost on top of each other.

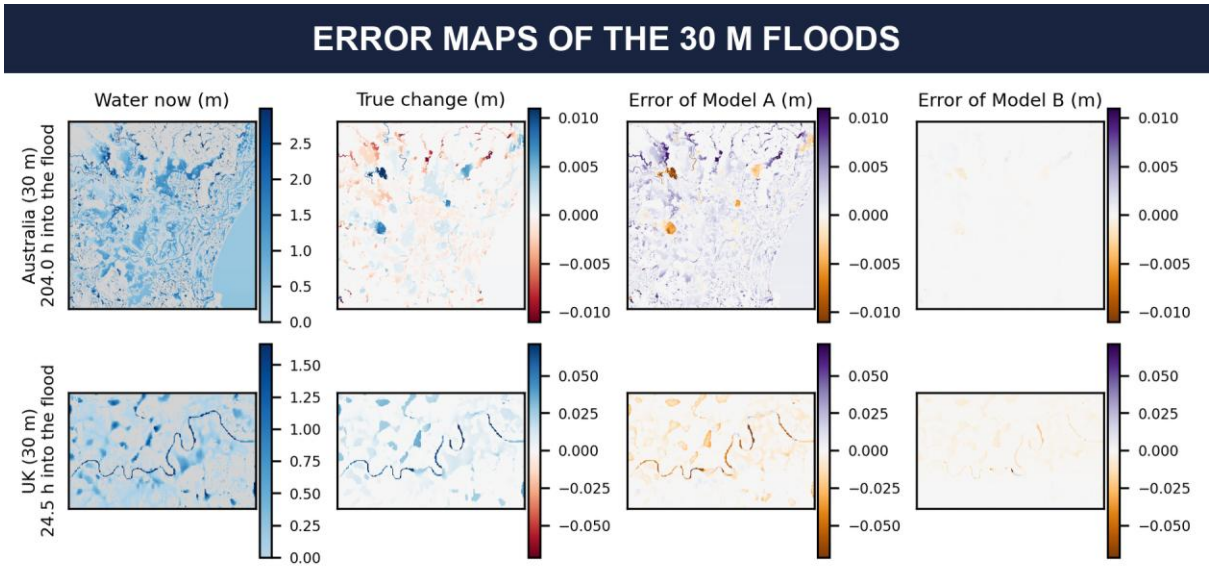


Figure 5: Error maps of the two 30 m floods at the moment of fastest change. The columns are the water now, the true change in 30 minutes, and the error of Model A and of Model B. Purple means too much water was forecast and orange means too little. The last three columns share one colour scale in each row.

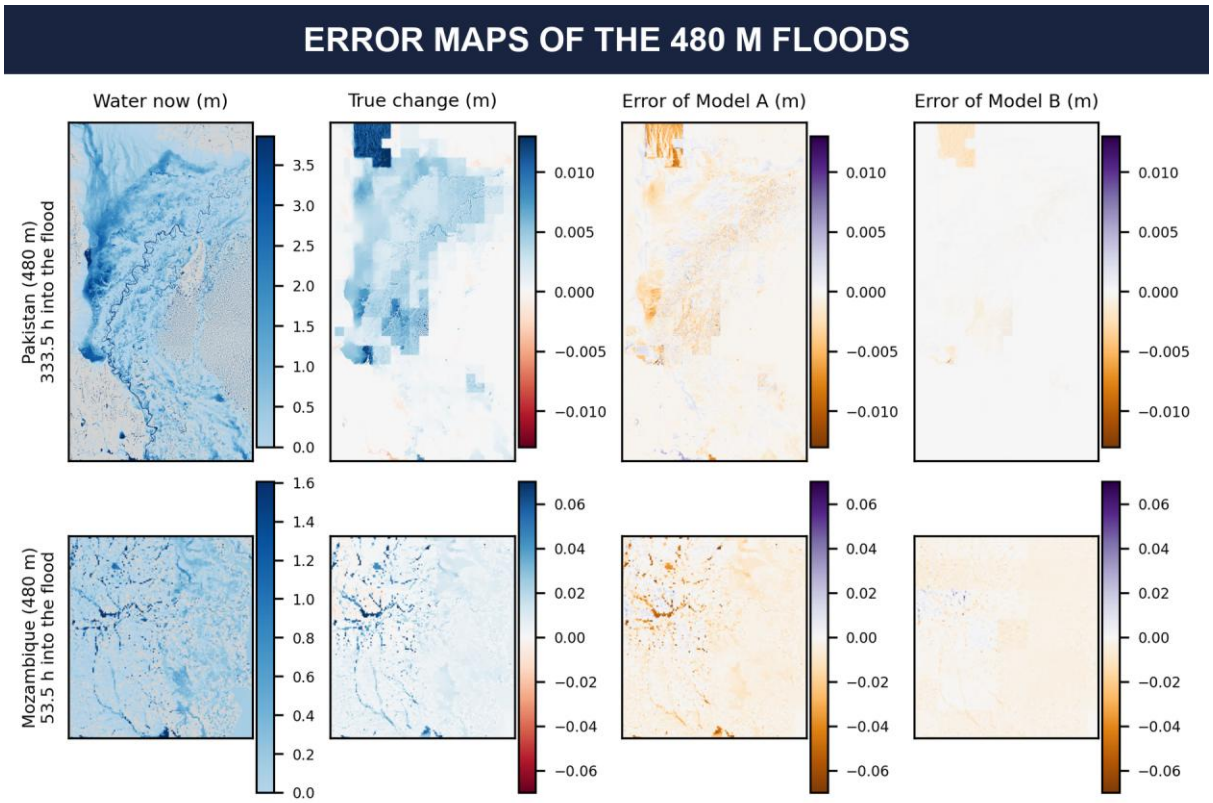


Figure 6: The same error maps for the two 480 m floods.

Figures 5 and 6 make the picture clear. The errors of Model A are strong along the rivers and at the blocks of heavy rain, while the errors of Model B are almost invisible at the same colour scale.

4.2 The water-balance rule

Before using the rule we checked how well the data obey the water balance. Over a whole flood the water added differed from the rain by up to about 5% (the ratio was 0.999 for Australia, 0.946 for the UK, 1.028 for Pakistan and 1.045 for Mozambique). The dataset paper says that water can leave freely at the map edge in three of the floods, and that the Pakistan simulation lets the Indus River flow in at the edge of the map. In 73 training steps of the Australian flood the average depth even fell, which rain alone cannot do. So the law holds closely but not exactly. Figure 7 shows the check.

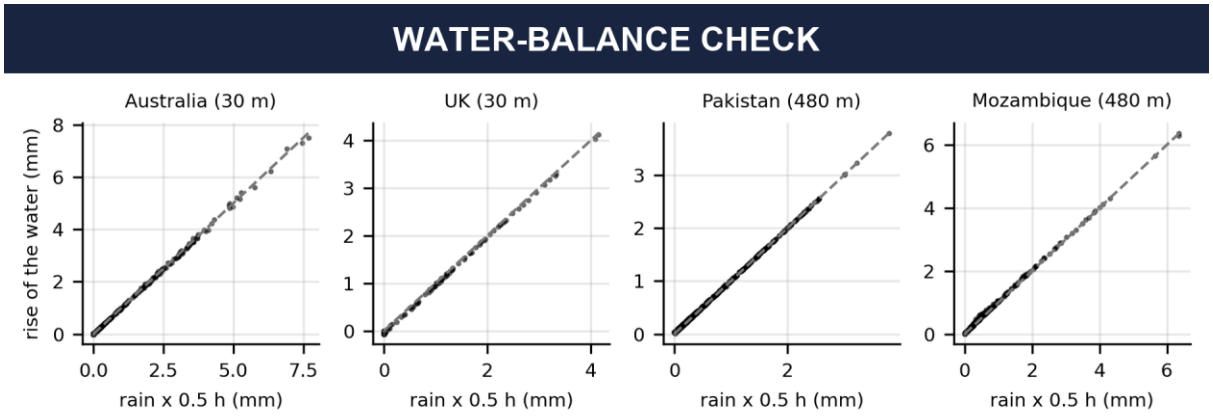


Figure 7: Water-balance check. Each dot is one 30-minute step. Across: rain times half an hour. Up: how much the water rose on average. The dashed line is the law.

The rise matches the rain of the same 30 minutes with a correlation of 1.00, but the rule can only use the rain of the last 30 minutes as its guess for the next 30 minutes, because future rain is unknown. The rise correlates less with the rain of the previous 30 minutes (0.925 to 0.969). The rule helped Model A in every test and halved the Australian failure (−1.449 to −0.710). On Model B it changed the score by between −0.007 and +0.001 (Table 6), and on copying alone it gave only 0.000 to 0.142. So the rule corrected a weakness of Model A, but it does not add skill to a good forecast. In our final system we keep it as an optional physics check, and Model B does not need it.

4.3 Experiment set II: comparison of architectures

Table 7 and Figure 8 give the skill of the simple forecasts and of the networks on the never-seen floods. The values are the mean over three folds with the hybrid rain input, where 0 means copying the last frame. Here the forecast covers five maps, from 30 minutes to 2.5 hours ahead, and only windows with water are scored. These numbers cannot be compared with Table 6. In particular, the linear model of the depth history in the first row is scored over all five lead times, while the trend forecast of Table 6 is scored at 30 minutes only and is damped by its fitted β.

Table 7: Skill on the never-seen flood against copying the last frame, mean of three folds (hybrid rain, one run per fold). The first four rows are fitted baselines, and the networks below them used the fixed 30-pass recipe. The two linear rows are ridge regressions on rescaled inputs. The first runs of the original CNN-LSTM on the UK were stopped by a faulty safety check and were repeated with identical settings and the check switched off. The seed-to-seed standard deviation is about 0.01. Models A and B of the notebook are not part of this table.

Forecast	E60 (UK, 60 m)	E30 (UK, 30 m)	E480 (Mozambique)
Linear model of the depth history (fitted)	0.818	0.798	0.790
Linear model with rain, state and trend	0.820	0.797	0.793
Two-feature lookup table	0.786	0.763	0.737
Per-pixel MLP	0.846	0.854	0.829
Original CNN-LSTM	0.187	0.188	0.061
CNN with 1 × 1 kernel (per pixel)	0.848	0.835	0.818
Local CNN (5 × 5, 4,101 parameters)	0.841	0.830	0.807
CNN with 11 × 11 kernel	0.841	0.831	0.793
Small ConvLSTM (4,741 parameters)	0.866	0.840	0.811
Heavy ConvLSTM (652,918 parameters)	0.840	not run	0.802

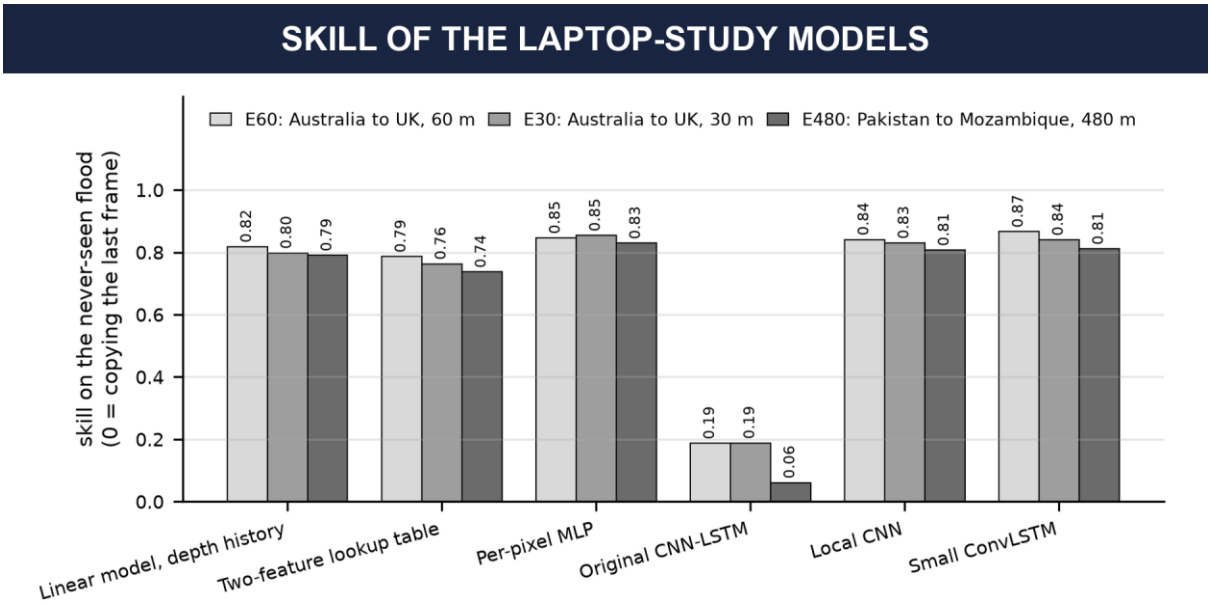


Figure 8: Selected rows of Table 7 as a chart (skill on the never-seen flood).

The local CNN and the small ConvLSTM reach 0.81 to 0.87, while the original CNN-LSTM reaches only 0.06 to 0.19. A table with only two inputs per cell, the current depth and the last change, already reaches 0.74 to 0.79, and a fitted linear model of the depth history reaches 0.79 to 0.82. A per-pixel MLP, which sees each cell's own history and a 15 by 15 average of the depth and of the last change around it, reaches 0.83 to 0.85. So the local CNN and the small ConvLSTM add only about 0.01 to 0.05 to what a linear model of the depth history already gives. The other rain settings show the same picture. The two networks are above a linear model fitted in the same way in 15 of the 16 cells (hypothesis H2 in Table 12), but in 7 of those cells by less than 0.02. Looking at neighbouring pixels does not help, because kernels of size 1, 5 and 11 score within 0.025 of each other and the version with the 1 × 1 kernel, which sees one cell only, is never the worst. To a first

approximation, the forecast is a simple function of the recent depth history of each cell, and most of it is linear.

Does recurrence help?

With the 30-pass recipe the small ConvLSTM was ahead of the local CNN on the UK at 60 m by +0.025 (95% interval +0.004 to +0.043) and level on Mozambique. But the plain CNN was slightly undertrained. When both were trained for 90 passes, the difference was +0.004 (−0.021 to +0.027) on the UK and −0.014 (−0.029 to +0.006) on Mozambique. Both intervals stay inside the band of ±0.03 that we fixed in advance as our meaning of equivalent. So at equal training length the two models are level. A ConvLSTM about 140 times larger scored no better (0.840 and 0.802).

The ConvLSTM also did better than the local CNN in two other ways, although we did not test whether recurrence is the reason. First, Table 8 shows that under the 30-pass recipe the ConvLSTM loses skill more slowly as the forecast reaches further ahead. It is behind the local CNN at 30 minutes and ahead of it at 2.5 hours. We did not check whether this survives equal training length. Second, when the start of a flood was hidden from training, the ConvLSTM stayed stable while the plain CNN failed (Section 4.8).

Table 8: Skill on the never-seen flood at each forecast time, mean of three folds (30-pass recipe, hybrid rain).

Test	Model	30 min	1 h	1.5 h	2 h	2.5 h
E60 (UK, 60 m)	Local CNN	0.930	0.900	0.869	0.841	0.817
	Small ConvLSTM	0.926	0.909	0.887	0.867	0.847
E30 (UK, 30 m)	Local CNN	0.927	0.895	0.861	0.831	0.804
	Small ConvLSTM	0.899	0.883	0.861	0.841	0.822
E480 (Mozambique)	Local CNN	0.918	0.880	0.841	0.807	0.778
	Small ConvLSTM	0.899	0.873	0.841	0.812	0.785

Why the original model failed

The original CNN-LSTM squeezes each frame through one dense layer of 128 numbers, and its depth input was scaled by 5 m, so a change of a millimetre is a tiny number. Table 9 shows that it ignores the depth history completely, which explains its low skill.

4.4 What the models use

In the test-time ablations we change one input of a trained model and see how much the score falls. “Rain removed” sets the rain to zero. “History frozen” replaces every earlier depth map by the last one, so no change over time is visible. “History shuffled” puts the four earlier depth maps in reverse order. In the training-time ablations we train a model without one ingredient.

Table 9: Skill on the never-seen flood when one input is changed at test time (hybrid rain, mean of three folds).

Model	Test	Normal	Rain removed	Terrain removed	History frozen	History shuffled
Original CNN-LSTM	E60	0.187	0.003	0.181	0.187	0.187
Local CNN	E60	0.841	0.833	0.841	−0.006	−3.037
Small ConvLSTM	E60	0.866	0.857	0.865	0.020	−2.649
Local CNN	E480	0.807	0.790	0.807	−0.008	−3.666
Small ConvLSTM	E480	0.811	0.791	0.811	−0.007	−2.929
Local CNN trained without the depth history: E60 0.172, E30 0.155, E480 0.086						

The networks use the depth history and almost nothing else. Freezing the history takes the skill to about zero, and shuffling it makes the forecast strongly worse than copying. For the four networks in Table 9 other than the original model, the rain is worth 0.01 to 0.02, and the terrain is worth 0.006 or less for all five. A network trained without the depth history falls from 0.84, 0.83 and 0.81 to 0.17, 0.16 and 0.09. The original model is the only one that depends on the rain instead of the history. This is why an early conclusion of our project, that rainfall drives the forecast, holds only for the original model.

The rain setting made a small difference (Table 10). The partly future rain of the original input was worth 0.01 to 0.025 over strictly past rain for the local CNN, and knowing the true future rain gave another 0.02 to 0.03.

Table 10: Skill on the never-seen flood for the three rain settings (mean of three folds).

Local CNN, rain setting	E60	E30	E480
Causal (past rain only)	0.822	0.805	0.794
Hybrid (the original input)	0.841	0.830	0.807
Forced (future rain also given)	0.866	0.851	0.834

4.5 Optimisation and training experiments

Table 11 shows what happened when we changed the optimiser settings, the training length and the amount of data. All rows use the hybrid rain input and give the never-seen skill, averaged over three folds.

Table 11: Effect of the optimiser setting, the training length and the amount of data on the never-seen skill.

Setting	Model	E60	E30	E480
Fixed recipe (batch 1, 30 passes)	Local CNN	0.841	0.830	0.807
Batch 8, learning rate 3×10^{-3}	Local CNN	0.829	0.822	0.798
90 passes instead of 30	Local CNN	0.862	0.838	0.831
600 training windows instead of 300	Local CNN	0.861	0.850	0.833
Fixed recipe	Small ConvLSTM	0.866	0.840	0.811
Batch 8, learning rate 3×10^{-3}	Small ConvLSTM	0.853	not run	0.785
90 passes instead of 30	Small ConvLSTM	0.866	not run	0.818

- A larger batch with a higher learning rate did not help. It lowered the score by 0.008 to 0.026, so this one change of step size did not help. We did not test other optimisers or other learning rates.
- Longer training helped the plain CNN by 0.008 to 0.024. The ConvLSTM gained almost nothing, so at equal length the two are level.
- Doubling the number of windows also helped the plain CNN, but mostly because it then made twice as many optimiser steps. When we held the number of steps equal, doubling the data changed the skill by only −0.002 to +0.008. Between 300 and 600 windows the plain CNN is limited by the amount of training, not by the number of windows.
- Five seeds of the local CNN gave a standard deviation of 0.006 to 0.010 on the never-seen flood (taken in each experiment and averaged over its three folds), and the largest gap between two seeds was 0.029. Differences smaller than about 0.02 are therefore not resolvable. On hidden Pakistan tiles the plain CNN was unstable (a standard deviation of 0.33 over five seeds), while the ConvLSTM, with one run per fold, scored between 0.83 and 0.86 on the three folds.
- A per-pixel MLP came within about 0.05 of its best score with only 25 to 50 training windows.

In the Colab notebook we used early stopping and kept the weights with the lowest validation error. For the 30 m Model A the validation error was lowest after the second pass and then rose while the training error kept falling, which is a sign of overfitting to the training period. For Model B the starting error was already low, so training gave only small gains.

4.6 Outcome of the pre-registered hypotheses

Table 12 gives the outcome of the six hypotheses that we wrote down before the final runs.

Table 12: Outcome of the pre-registered hypotheses for the never-seen flood and the held-out tiles.

	Hypothesis	Outcome
H1	The local CNN and the small ConvLSTM beat copying (both intervals above 0)	Held in 13 of 16 cells. The three exceptions are the local CNN on E480, where the intervals on the hidden Pakistan tiles include 0
H2	They beat the best linear baseline	Held in 15 of 16 cells by the planned test (mean skill above the baseline). The gain is 0.02 or more in 8 cells and below 0.02, which is within seed noise, in 7 cells. In 1 cell, the ConvLSTM with forced rain on E480, the linear model is better by 0.046 (Section 4.3)
H3	The depth history matters	Held in all three experiments (a loss of 0.67 to 0.72 without it)
H4	Recurrence matters	Mixed. At the planned 30 passes the ConvLSTM was ahead on UK 60 m (+0.025) and level on Mozambique. It was not supported at equal training length (90 passes, a check added after the first results, Section 4.3)
H5	More rain information helps (causal, hybrid, forced)	Held for the local CNN in all three experiments, not for the ConvLSTM, whose forced-rain input layout seems to have been a handicap (giving the future rain as extra channels raised its score by 0.03 to 0.06)
H6	A stricter test gives a similar score (within 0.05)	Held in 2 of 5 cells (Section 4.8)

4.7 Severity, rules and Grad-CAM

Capsule network against a plain layer

On Australia at 30 m with the depth-based label, the capsule head reached 67.8% on the natural class mix and the dense control 60.2%, a gap of +0.076 (interval +0.051 to +0.101). With the eight rotations and flips of each patch used as augmentation, the two reached 80.3% and 79.8%, and the gap shrank to +0.005 (−0.014 to +0.022). Both were far below a trivial rule that thresholds the last observed frame (97.4%) or the model's own forecast (98.6% to 98.7%). The rule layer of the laptop study lowered the accuracy, for example from 80.3% to 74.1%. The rule layer of the laptop study and the rules of the notebook are different rules that are used with different labels, so their results cannot be compared directly.

Colab results with the new label

Table 13 shows the capsule network on all four floods. The class of the forecast map alone is right about 99% of the time, because the rating in 30 minutes is almost the same as the rating now. The classifier is still of interest because a rating is easy for a non-expert to read, but it adds little to the forecast map here. The rules raised the accuracy in all four floods. This came mostly from rule 1 in Australia and Mozambique and from rule 2 in the UK and Mozambique. In three floods the rules also copied the class of the forecast map (rule 3 changed 21, 21 and 10 tiles in Australia, the UK and Pakistan, and in Pakistan it made 10 of the 17 changes and rule 2 the other 7). In the Pakistan test 698 of the 1,000 tiles are severe, so always answering "severe" would already be right 69.8% of the time. A 64 × 64

tile covers about 2 km at 30 m but about 30 km at 480 m, so the accuracies of the two models should not be compared. The 1,000 tiles of each flood overlap in time and are not independent. Figure 9 shows the confusion matrices.

Table 13: Severity accuracy on 1,000 test tiles per flood (Colab notebook, new label).

Test flood	Capsule alone	Capsule + rules	Forecast-map class alone
Australia (30 m)	60.9%	67.9%	99.8%
UK (30 m)	50.0%	56.1%	99.1%
Pakistan (480 m)	87.4%	89.1%	99.9%
Mozambique (480 m)	63.5%	70.7%	99.5%

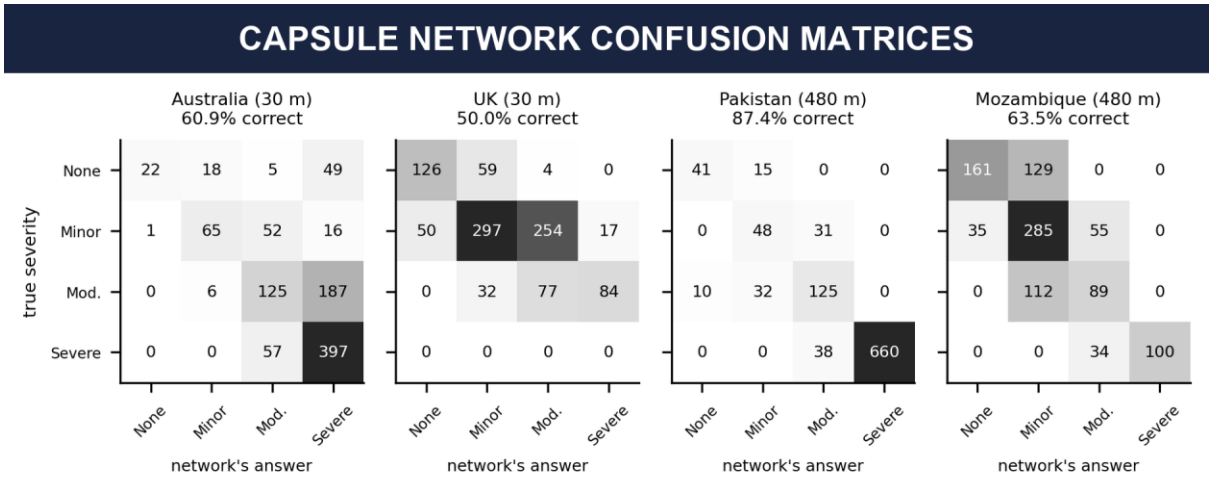


Figure 9: Confusion matrices of the capsule network. Rows are the true rating and columns are the network's answer.

Grad-CAM

Figure 10 shows the heat maps of the forecaster. In Australia and the UK the bright areas follow the river channels and flood edges, in Pakistan they match large blocks, which we think are the blocks of heavy rain, and in Mozambique they follow the drainage lines. These are places where the water is changing, which is sensible. In the laptop study, deleting the top 10% of the heat map changed the forecast more than deleting random or displaced cells in 84% to 92% of the windows, so the depth heat maps pass that test. For the severity class the heat maps were weak (the top cells beat both controls in only 55% of windows for the capsule head), and the maps of two networks trained with different seeds agreed with a rank correlation of only 0.17. We therefore treat the capsule heat maps as illustrations, not as proof.

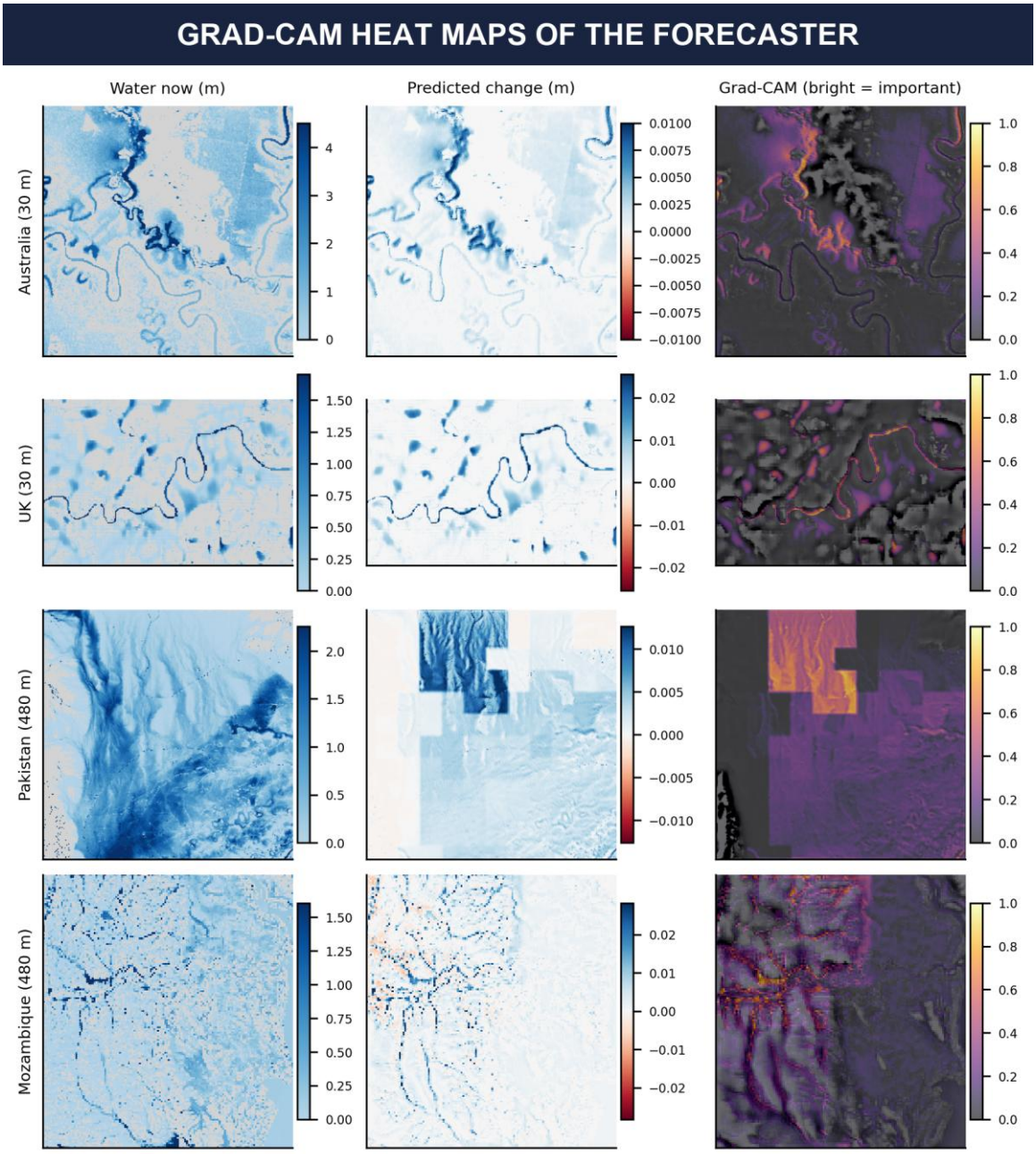


Figure 10: Grad-CAM of the forecaster (Model A). The columns are the water now, the predicted change and the heat map on top of the local ground height. For the large maps we zoom into the busiest window, because a whole map does not fit in the GPU memory.

4.8 Where the models fail, and limits of this study

- When we also hid a block of time from training (the strict split), the same-region score of the plain CNN on Pakistan fell from 0.430 to −6.56. Six of 480 windows, all at the very start of the flood, carried 98% of the error, and the failure repeated across five seeds and a second recipe. The ConvLSTM stayed at 0.742 and the lookup table at 0.66, while the per-pixel MLP also failed. Training on the small Mozambique flood and testing on Pakistan failed in the same way for the plain networks. A model that has not seen the start of a flood should not be trusted on one.

- Every region has one flood, and E60 and E30 share the UK flood. So there are only two independent train-and-test pairs, even though our tables have many rows.
- The floods are simulations, so the models learn to imitate the simulator. They have not been tested against observed flooding.
- The Colab results are single runs. The two experiment sets also use different protocols, whole maps for 30 minutes against patches for 2.5 hours, so their numbers cannot be compared.
- The capsule network and the Grad-CAM pictures of the notebook were built on Model A, and we did not rebuild them on Model B.

5. Conclusion

We built a CNN-ConvLSTM forecaster for flood depth maps, added a capsule severity network, Grad-CAM and written rules, and tested them on later parts of the training floods and on floods from other countries. The main lesson is about how to judge such a model. Model A beat the easiest reference, copying the last map, and looked successful. A second simple reference that continues the last change was far better, and Model A scored well below it. Model B, which starts from this trend, matched it but did not beat it.

The larger laptop study showed the same picture from another side. The depth history carries nearly all of the skill. A fitted linear model of it reaches 0.79 to 0.82, against 0.81 to 0.87 for the best networks, and neighbouring pixels add nothing. At equal training length the ConvLSTM was no more accurate than a plain CNN, but it was more stable when the start of a flood was hidden, and its skill fell more slowly with the forecast time under the 30-pass recipe. A larger batch with a higher learning rate did not help, and longer training helped a little. Differences below 0.02 are within seed noise. The capsule network did not beat a plain layer once augmentation was used, and neither beat a trivial rule on the severity task. The plain CNN also fails on the very start of a flood that was missing from training, while the ConvLSTM and the lookup table did not.

Future work

The most useful next step is to forecast further ahead, for 6 to 24 hours, by feeding each forecast back as input. This is where a trend should get worse and the memory of the LSTM has a real chance to matter, so it is the fair test of the CNN-LSTM. We first have to decide which rain to assume for the future steps, and the trend forecast should be carried forward in the same way for comparison. We would also like to train Model A with only the centimetre scaling and Model B without it, which would separate the two changes that we made at once. Other steps are to rebuild the capsule network and Grad-CAM on Model B and compare the capsule network with a plain layer on the new severity label at 3 to 6 hours ahead, and to repeat the Colab runs with several seeds.

6. **References**

[1] Q. Xu, Y. Shi, J. Zhao, and X. X. Zhu, “FloodCastBench: A large-scale dataset and foundation models for flood modeling and forecasting,” *Scientific Data*, vol. 12, article 431, 2025. DOI: 10.1038/s41597-025-04725-2

[2] A. Mosavi, P. Ozturk, and K.-W. Chau, “Flood prediction using machine learning models: Literature review,” *Water*, vol. 10, no. 11, article 1536, 2018. DOI: 10.3390/w10111536

[3] R. Bentivoglio, E. Isufi, S. N. Jonkman, and R. Taormina, “Deep learning methods for flood mapping: a review of existing applications and future research directions,” *Hydrology and Earth System Sciences*, vol. 26, no. 16, pp. 4345-4378, 2022. DOI: 10.5194/hess-26-4345-2022

[4] F. Kratzert, D. Klotz, C. Brenner, K. Schulz, and M. Herrnegger, “Rainfall-runoff modelling using Long Short-Term Memory (LSTM) networks,” *Hydrology and Earth System Sciences*, vol. 22, no. 11, pp. 6005-6022, 2018. DOI: 10.5194/hess-22-6005-2018

[5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, 1998. DOI: 10.1109/5.726791

[6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735-1780, 1997. DOI: 10.1162/neco.1997.9.8.1735

[7] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W.-K. Wong, and W.-C. Woo, “Convolutional LSTM network: A machine learning approach for precipitation nowcasting,” in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, 2015, pp. 802-810. arXiv:1506.04214

[8] Y. Wang, M. Long, J. Wang, Z. Gao, and P. S. Yu, “PredRNN: Recurrent neural networks for predictive learning using spatiotemporal LSTMs,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, pp. 879-888. https://proceedings.neurips.cc/paper/2017/hash/e5f6ad6ce374177eef023bf5d0c018b6-Abstract.html

[9] Q. Xu, Y. Shi, J. L. Bamber, C. Ouyang, and X. X. Zhu, “Large-scale flood modeling and forecasting with FloodCast,” *Water Research*, vol. 264, article 122162, 2024. DOI: 10.1016/j.watres.2024.122162

[10] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier neural operator for parametric partial differential equations,” in *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.08895

[11] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*, Lecture Notes in Computer Science, vol. 9351, 2015, pp. 234-241. DOI: 10.1007/978-3-319-24574-4_28

[12] J. Xiao, Z. Wang, Y. Liao, Y. Yi, L. Zheng, B. Yang, H. Yu, X. Li, N. Hu, and C. Lai, “A ConvLSTM-based model for urban flood prediction under dynamic rainfall patterns and exploration on its extrapolation capability,” *International Journal of Disaster Risk Science*, vol. 16, no. 6, pp. 1057-1073, 2025. DOI: 10.1007/s13753-025-00685-8

[13] X. Xiang, S. Guo, C. Li, B. Sun, and Z. Liang, “Deep learning model for flood probabilistic forecasting considering spatiotemporal rainfall distribution and hydrologic uncertainty,” *Journal of Hydrology*, vol. 655, article 132879, 2025. DOI: 10.1016/j.jhydrol.2025.132879

[14] S. Sabour, N. Frosst, and G. E. Hinton, “Dynamic routing between capsules,” in *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017, pp. 3856-3866. arXiv:1710.09829

[15] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618-626. DOI: 10.1109/ICCV.2017.74

[16] A. d’Avila Garcez and L. C. Lamb, “Neurosymbolic AI: The 3rd wave,” *Artificial Intelligence Review*, vol. 56, no. 11, pp. 12387-12406, 2023. DOI: 10.1007/s10462-023-10448-w

[17] A. Karpatne, G. Atluri, J. H. Faghmous, M. Steinbach, A. Banerjee, A. Ganguly, S. Shekhar, N. Samatova, and V. Kumar, “Theory-guided data science: A new paradigm for scientific discovery from data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 10, pp. 2318-2331, 2017. DOI: 10.1109/TKDE.2017.2720168

[18] R. J. Hyndman and A. B. Koehler, “Another look at measures of forecast accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679-688, 2006. DOI: 10.1016/j.ijforecast.2006.03.001

[19] A. H. Murphy, “Skill scores based on the mean square error and their relationships to the correlation coefficient,” *Monthly Weather Review*, vol. 116, no. 12, pp. 2417-2424, 1988. DOI: 10.1175/1520-0493(1988)116<2417:SSBOTM>2.0.CO;2

[20] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “Statistical and machine learning forecasting methods: Concerns and ways forward,” *PLOS ONE*, vol. 13, no. 3, article e0194889, 2018. DOI: 10.1371/journal.pone.0194889

[21] A. Zeng, M. Chen, L. Zhang, and Q. Xu, “Are transformers effective for time series forecasting?,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, 2023, pp. 11121-11128. DOI: 10.1609/aaai.v37i9.26317

[22] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980